

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Вариант 7. Вывод графического представления автомата.

Студент гр. 4343

Зеров М.М.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

Задание 1

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст ($T, 1 \leq |T| \leq 100000$).

Вторая - число n ($1 \leq n \leq 3000$), каждая следующая из n строк содержит шаблон из набора $= \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Выход:

Все вхождения образцов из P в T .

Каждое вхождение образца в текст представить в виде двух чисел - i p

Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p
(нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Тестовые данные

№ Теста

1

Входные данные

NTAG 3 TAGT TAG T

Выходные данные

2 2 2 3

Задание 2

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с *джокером*.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблону образцу PP необходимо найти все вхождения PP в текст TT .

Например, образец $ab??c?ab??c?$ с джокером $??$ встречается дважды в тексте $xabvccbababcsaxxabvccbababcsax$.

Символ джокер не входит в алфавит, символы которого используются в TT . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы.

Все строки содержат символы из алфавита $\{A,C,G,T,N\}$

Вход:

Текст ($T, 1 \leq |T| \leq 1000000$)

Шаблон ($P, 1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Тестовые данные

№ Теста

1

Входные данные
ACTANCA
A\$\$\$
\$
Выходные данные
1

Описание алгоритма:

Задача 1:

Шаг 1. Построение бора

Из всех образцов строится бор (trie).

Каждая вершина бора соответствует некоторому префиксу одного или нескольких образцов.

Вершины, в которых заканчивается образец, помечаются как терминальные с номером этого образца.

Сложность: $O(m)$, где m — суммарная длина всех образцов.

Шаг 2. Построение суффиксных ссылок

Для каждой вершины v вычисляется $fail[v]$ — ссылка на вершину, соответствующую самому длинному собственному суффиксу строки, которую представляет v , являющемуся префиксом какого-то образца.

Вычисление выполняется обходом в ширину (BFS) от корня.

Формула: $fail[u] = nxt[fail[v]][c]$, где u — сын v по символу c .

Для корня и вершин первого уровня $fail = 0$.

Дополнительно выполняется замыкание переходов:

если у v нет перехода по символу c , то $nxt[v][c] = nxt[fail[v]][c]$.

Шаг 3. Построение конечных (сжатых) ссылок

Для каждой вершины v вычисляется $outLink[v]$ — ближайшая вершина по цепочке $fail$ -ссылок, в которой заканчивается хотя бы один образец.

Конечные ссылки позволяют при поиске за один проход собрать все образцы, заканчивающиеся в текущей позиции текста, без ручного обхода всей $fail$ -цепочки.

Использование автомата

Проходим текст посимвольно. Из текущего состояния u переходим по функции $\delta(u, c) = nxt[u][c]$. Попад в новое состояние v , проходим

по цепочке конечных ссылок:

```
int x = v;  
while (x != -1) {  
    для каждого образца p в out[x]:  
        зафиксировать вхождение p на позиции i - len[p] + 1;  
        x = outLink[x];  
}
```

Сложность поиска: $O(|T| + \text{число вхождений})$.

Задача 2:

Шаг 1. Разбиение образца на сегменты

Образец P разбивается по джокерам на сегменты без джокеров.

Для каждого сегмента сохраняется его смещение в P:

$\text{segs} = \{(\text{offset_1}, Q_1), \dots, (\text{offset_k}, Q_k)\}$

Шаг 2. Множественный поиск сегментов

К множеству сегментов применяется алгоритм Ахо-Корасик

(шаги 1-3 из задачи 1). При поиске в тексте для каждого найденного

сегмента Q_j в позиции i вычисляется возможный старт образца:

$\text{start} = i - |Q_j| + 1 - \text{offset_j}$

И увеличивается счётчик $\text{cnt}[\text{start}]$.

Шаг 3. Определение вхождений

Позиция i является вхождением образца P в текст T тогда и только

тогда, когда $\text{cnt}[i] == k$, где k — количество сегментов.

Это означает, что все безмасочные куски P совпали одновременно

при наложении образца со стартовой позицией i .

Сложность: $O(|T| + |P| + a)$, где a — число вхождений сегментов.

Пример графического представления автомата в соответствии с вариантом:

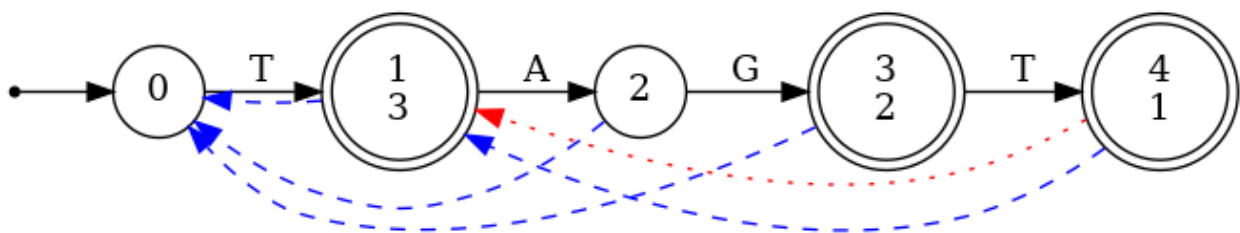


Рисунок 1 – графическое представление автомата

Кружок		Состояние автомата
Цифра сверху	Номер состояния	
Цифры снизу	Номера образцов, заканчивающихся здесь	
Двойной кружок	Терминальное состояние	
Чёрная стрелка с буквой	Переход бора по символу	
Синий пунктир	Суффиксная ссылка	
Красный пунктир	Конечная ссылка outLink	

Чёрная точка слева	Начало
-----------------------	--------

Оценка сложности:

Задача 1:

Построение бора: $O(m)$

Построение суффиксных ссылок (BFS): $O(m * k)$,

где $k = |\text{алфавит}| = 5$

Построение конечных ссылок: $O(m)$

Поиск в тексте: $O(|T| + a)$

Общая сложность: $O(m + |T| + a)$

где m — суммарная длина образцов,

$|T|$ — длина текста,

a — количество найденных вхождений.

Задача 2:

Разбиение образца: $O(|P|)$

Построение автомата: $O(m + k)$, $m = |P|$

Поиск всех сегментов: $O(|T| + a)$

Проверка счётчиков: $O(|T|)$

Общая сложность: $O(|T| + |P| + a)$

Тесты:

ТЕСТЫ: Алгоритм Ахо-Корасик множественный поиск
[=====] Running 8 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 8 tests from ACTest
[RUN] ACTest.SinglePattern
[1] Один образец в тексте

ВХОДНЫЕ ДАННЫЕ:

Текст: "АСАСАСА"
Образцов: 1
Образец #1: "АСА"

РЕЗУЛЬТАТ: (1,1), (3,1), (5,1)
ОК (найдено 3 вхождений)
[ОК] ACTest.SinglePattern (0 ms)
[RUN] ACTest.MultiplePatterns
[2] Несколько образцов

ВХОДНЫЕ ДАННЫЕ:

Текст: "TAGCTN"
Образцы:
#1: "ТА"
#2: "ТАГ"
#3: "GC"
#4: "СТ"

РЕЗУЛЬТАТ: (1,1), (1,2), (3,3), (4,4)
ОК (найдено 4 вхождений)
[ОК] ACTest.MultiplePatterns (0 ms)
[RUN] ACTest.NoMatches
[3] Образец не найден

ВХОДНЫЕ ДАННЫЕ:

Текст: "АААА"
Образцов: 1
Образец #1: "ССС"

РЕЗУЛЬТАТ: (пусто)
ОК (вхождений не найдено)
[ОК] ACTest.NoMatches (0 ms)
[RUN] ACTest.OverlappingPatterns
[4] Перекрывающиеся вхождения

ВХОДНЫЕ ДАННЫЕ:

Текст: "ААААА"
Образцы: "АА", "ААА"

РЕЗУЛЬТАТ: (1,1), (1,2), (2,1), (2,2), (3,1), (3,2), (4,1)
ОК (найдено 7 вхождений)
[ОК] ACTest.OverlappingPatterns (0 ms)
[RUN] ACTest.SuffixPattern
[5] Один образец — суффикс другого (проверка outLink)

ВХОДНЫЕ ДАННЫЕ:

Текст: "TAGT"

Образцы:

#1: "TAGT"

#2: "AGT"

#3: "GT"

#4: "T"

РЕЗУЛЬТАТ: (1,1), (1,4), (2,2), (3,3), (4,4)

ОК (найдено 5 вхождений)

[OK] ACTest.SuffixPattern (0 ms)

[RUN] ACTest.EmptyText

[6] Пустой текст

ВХОДНЫЕ ДАННЫЕ:

Текст: пустой

Образец #1: "ACG"

РЕЗУЛЬТАТ: (пусто)

ОК (вхождений нет)

[OK] ACTest.EmptyText (0 ms)

[RUN] ACTest.PatternEqualsText

[7] Образец совпадает с текстом

ВХОДНЫЕ ДАННЫЕ:

Текст: "ACG"

Образец #1: "ACG"

РЕЗУЛЬТАТ: (1,1)

ОК (найдено 1 вхождений)

[OK] ACTest.PatternEqualsText (0 ms)

[RUN] ACTest.MultipleSuffixes

[8] Цепочка суффиксов A, AA, AAA, AAAA в AAAA

ВХОДНЫЕ ДАННЫЕ:

Текст: "AAAA"

Образцы:

#1: "A"

#2: "AA"

#3: "AAA"

#4: "AAAA"

РЕЗУЛЬТАТ: (1,1), (1,2), (1,3), (1,4), (2,1), (2,2), (2,3), (3,1), (3,2), (4,1)

ОК (найдено 10 вхождений)

[OK] ACTest.MultipleSuffixes (0 ms)

[-----] 8 tests from ACTest (0 ms total)

[-----] Global test environment tear-down

[=====] 8 tests from 1 test suite ran. (0 ms total)

[PASSED] 8 tests.

РЕЗУЛЬТАТЫ

Пройдено: 8

Не пройдено: 0

ТЕСТЫ: Поиск образца с джокером
[=====] Running 8 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 8 tests from WildTest
[RUN] WildTest.ExampleFromTask
[1] Пример из условия

ВХОДНЫЕ ДАННЫЕ:

Текст: "АСТАНСА"
Образец: "A\$\$A\$"
Джокер: '\$'

РЕЗУЛЬТАТ: 1

ОК (найдено 1 вхождений)
[ОК] WildTest.ExampleFromTask (0 ms)
[RUN] WildTest.NoWildcard
[2] Образец без джокеров (обычный поиск)

ВХОДНЫЕ ДАННЫЕ:

Текст: "АСАСАСА"
Образец: "АСА"
Джокер: '?'

РЕЗУЛЬТАТ: 1, 3, 5

ОК (найдено 3 вхождений)
[ОК] WildTest.NoWildcard (0 ms)
[RUN] WildTest.WildcardAtStart
[3] Джокер в начале образца

ВХОДНЫЕ ДАННЫЕ:

Текст: "NACG"
Образец: "?ACG"
Джокер: '?'

РЕЗУЛЬТАТ: 1

ОК (найдено 1 вхождений)
[ОК] WildTest.WildcardAtStart (0 ms)
[RUN] WildTest.WildcardAtEnd
[4] Джокер в конце образца

ВХОДНЫЕ ДАННЫЕ:

Текст: "ACGN"
Образец: "ACG?"
Джокер: '?'

РЕЗУЛЬТАТ: 1

ОК (найдено 1 вхождений)
[ОК] WildTest.WildcardAtEnd (0 ms)
[RUN] WildTest.MultipleWildcards
[5] Несколько джокеров

ВХОДНЫЕ ДАННЫЕ:

Текст: "ACGACGACG"
Образец: "A?G"
Джокер: '?'

РЕЗУЛЬТАТ: 1, 4, 7
ОК (найдено 3 вхождений)
[OK] WildTest.MultipleWildcards (0 ms)
[RUN] WildTest.NoMatches
[6] Совпадений нет

ВХОДНЫЕ ДАННЫЕ:

Текст: "AAAA"
Образец: "C?C"
Джокер: "?"

РЕЗУЛЬТАТ: (пусто)
ОК (вхождений не найдено)
[OK] WildTest.NoMatches (0 ms)
[RUN] WildTest.PatternLongerThanText
[7] Образец длиннее текста

ВХОДНЫЕ ДАННЫЕ:

Текст: "ACG"
Образец: "ACGT"
Джокер: "?"

РЕЗУЛЬТАТ: (пусто)
ОК (вхождений нет)
[OK] WildTest.PatternLongerThanText (0 ms)
[RUN] WildTest.OnlyOneSegment
[8] Один сегмент (джокеры только в конце)

ВХОДНЫЕ ДАННЫЕ:

Текст: "ACGACG"
Образец: "ACG??"
Джокер: "?"

РЕЗУЛЬТАТ: 1
ОК (найдено 1 вхождений)
[OK] WildTest.OnlyOneSegment (0 ms)
[-----] 8 tests from WildTest (0 ms total)

[-----] Global test environment tear-down
[=====] 8 tests from 1 test suite ran. (0 ms total)
[PASSED] 8 tests.

РЕЗУЛЬТАТЫ

Пройдено: 8
Не пройдено: 0